LLARS - Script Creator & Framework Guide

Current Creator API Edition

Welcome to the current LLARS script creator documentation.

LLARS stands for Local Library of Automated Routines & Scripts.

This guide is for creators writing, converting, maintaining, or contributing scripts using the current Creator API. The modern framework keeps task-specific automation in the script while LLARS owns shared lifecycle, configuration, timing, focus safety, diagnostics, and cleanup.

Traditional AutoHotkey patterns are intentionally separated into the LLARS - Legacy Script Creator Guide. This guide teaches the current framework first and does not use old migration patterns as the default.

Table of Contents

1. Before You Start
2. How Modern LLARS Is Designed
3. Project Structure
4. Templates and the Script Generator
5. The Script Skeleton and Editing Markers
6. Config.ini and Typed Setup
7. Creator Configuration API
8. RunCount Scripts
9. The Run Context Object
10. RunCount Runtime Estimation
11. RuneScape Focus and Input Safety
12. Clicks and NaturalClick
13. Keyboard Input
14. Status and Activity Text
15. Sleeps and Random Sleep
16. Managed Callback Timers
17. Timer, MultiTimer, and UntilDone Scripts
18. Pixel and Color APIs
19. Optional Features and Dependencies
20. Developer Mode
21. Logging and Diagnostics
22. Testing a Script
23. Converting an Older Script
24. Creator API Reference
25. Framework Presence and Includes
26. Common Creator Mistakes
27. About LLARS
28. The Simple Version

1. Before You Start

- Use AutoHotkey v1.1.37.02. LLARS is not an AutoHotkey v2 framework.
- Keep the full LLARS project folder extracted and together while developing and testing.
- Start from the maintained template closest to the behavior you need.
- Use the Creator API for normal clicks, keys, configured hotkeys, sleeps, timers, pixel reads, and RunCount lifecycle.
- Keep the exact script editing markers intact. Runtime estimation and LLARS tooling use them.

> **Note:** The current project is intentionally API-first. Use traditional IniRead/Random/Send/SetTimer patterns only when the script truly needs lower-level AutoHotkey behavior, and use the separate Legacy Guide for those cases.

2. How Modern LLARS Is Designed

The central idea is simple: creator scripts describe what the RuneScape task does; Core owns the repetitive framework behavior around it.

Creator code normally owns

- The action sequence for the RuneScape task.
- Task-specific helper functions and state.
- Meaningful Config.ini section names and comments.
- Choices that are genuinely script-specific.

LLARS normally owns

- RunCount prompting, loop bookkeeping, progress, runtime estimation, completion, and cleanup.
- Timed-run prompting and shared timer GUI state.
- Config validation and typed setup metadata.
- RuneScape focus recovery before game-facing actions.
- Natural mouse movement and safe clicking.
- Managed callback timer registration and cleanup.
- Developer Mode diagnostics, framework logging, and shared control hotkeys.

> **Note:** A script should not recreate a framework feature just because AutoHotkey makes it possible. Use raw AHK when the behavior is genuinely task-specific, not as a replacement for an existing LLARS API.

3. Project Structure

```
LLARS\
|-- Core\
|   |-- LLARS.ahk
|   |-- LLARS_Core.ahk
|   |-- LLARS_CreatorAPI.ahk
|   |-- LLARS_GUI.ahk
|   |-- LLARS_Labels.ahk
|   `-- LLARS_Runtime.ahk
|
|-- Scripts\
|   `-- Your Script\
|       |-- Your Script.ahk
|       `-- Config.ini
|
|-- Templates\
```

```
|   |-- Script Generator.ahk
|   |-- RunCount\
|   |-- Timer\
|   |-- MultiTimer\
|   |-- Color\
|   `-- MultiColor\
|
|-- Documentation\
|-- LLARS Config.ini
`-- Script Selector.ahk
```

The script-local Config.ini belongs to one script. The root LLARS Config.ini contains shared settings such as LLARS control hotkeys, Random Sleep, and other framework-wide options.

4. Templates and the Script Generator

The recommended starting point is Templates\Script Generator.ahk. It creates a script folder from one of the maintained templates and preserves the framework bootstrap and include tail.

Template	Use it when
RunCount - Basic Loop	Every requested repetition follows the same main sequence.
RunCount - Advanced Loop	The first repetition differs from following repetitions.
Timer	One or more actions occur during a user-selected overall duration.
MultiTimer	Several independent timed actions run during the same overall duration.
Color Detection	A timed script reacts to one configured pixel/color state.
Multi Color Detection	A timed script recognizes multiple valid colors/states and can rotate action locations.

Manual template copying is still valid, but do not strip out the framework-presence functions, bootstrap includes, shared Start/End structure, or editing markers.

5. The Script Skeleton and Editing Markers

RunCount templates now have a very small creator-facing start path:

```
Start:

LLARS_RunCount(Func("Run"))

return
```

The task-specific code belongs between one exact begin marker and one exact end marker:

```
; ================================================================
; SCRIPT_EDIT_BEGIN_4C4C415253
; ================================================================

; CREATOR CODE

; ================================================================
; SCRIPT_EDIT_END_4C4C415253
; ================================================================
```

> **Note:** Do not rename, reformat, remove, or duplicate either marker. The RunCount estimator reads only this region, and migration/tooling also uses these boundaries.

6. Config.ini and Typed Setup

Config.ini is still the user-facing description of script settings. The modern difference is that creator code usually names a section and lets LLARS perform the reads, validation, randomization, and action.

Coordinate area

```
#Area to click in RuneScape.
[Action Location]
type=coordinate
xmin=
xmax=
ymin=
ymax=
```

Exact point

```
#Pixel to monitor.
[Pixel Coordinate]
type=coordinate
x=
y=
```

Hotkey and color

```
[Action Hotkey]
type=hotkey
hotkey=

[Target Color]
type=color
coordinate=Pixel Coordinate
color=0xFFFFFF
```

Sleep or timer range

```
[Sleep Short]
min=500
max=1000

[Action Timer]
min=15000
max=30000
```

Typed coordinate, hotkey, and color metadata drives the setup GUI, status, reset, and diagnostics. A color may add coordinate=<point section> so the GUI samples that saved pixel; sleep/timer ranges need no type line.

7. Creator Configuration API

Use behavior helpers first. Read raw values only when task-specific logic genuinely needs the value.

Need	Preferred call
Click a configured point/area	LLARS_Click("Action Location")
Press a configured hotkey	LLARS_PressHotkey("Action Hotkey")
Wait using a configured range	LLARS_Sleep("Sleep Short")
Read a point for custom math	LLARS_ConfigReadPoint("Section", x, y)
Read a min/max range	LLARS_ConfigReadRange("Section", minValue, maxValue)

Read a boolean option	LLARS_ConfigReadBool("Feature")
Read an integer	LLARS_ConfigReadInteger("Settings", "count", "", 1)
Read a number	LLARS_ConfigReadNumber("Settings", "offset", 0)
Read text	LLARS_ConfigReadString("Settings", "name", "", true)
Read an allowed choice	LLARS_ConfigReadChoice("Settings", "mode", "One|Two|Three")
Read a color	LLARS_ConfigReadColor("Target Color")

All config helpers default to the script-local Config.ini. Pass scope="shared" only when a script intentionally needs a value from the root LLARS Config.ini.

8. RunCount Scripts

For current RunCount scripts, LLARS owns the loop. The creator supplies a callback that represents one repetition.

```
Start:

LLARS_RunCount(Func("Run"))

return

Run(ctx)
{
    LLARS_SetStatus("Banking")
    LLARS_Click("Bank")
    LLARS_Sleep("Sleep Short")
    LLARS_PressHotkey("Bank Preset")
    LLARS_Sleep("Sleep Craft", true)
}
```

LLARS_RunCount() owns the run-count prompt, start validation, loop bookkeeping, progress display, runtime measurement, managed timer cleanup, normal completion, and error handling around the callback.

> **Note:** New RunCount scripts should not manually own Loop, runcount, firstrun, LLARS_BeginLoop(), LLARS_EndLoop(), or LLARS_RunComplete(). Those are compatibility/lower-level concepts documented in the Legacy Guide.

9. The Run Context Object

LLARS passes a context object to every RunCount callback:

Property	Meaning
ctx.Iteration	Current repetition number, starting at 1.
ctx.Total	Total runs requested by the user.
ctx.IsFirst	True only for repetition 1.
ctx.IsLast	True only for the final requested repetition.
ctx.Completed	How many repetitions were completed before the current one.
ctx.Remaining	How many repetitions remain after the current one.

First run differs

```
Run(ctx)
{
    if (ctx.IsFirst)
    {
        LLARS_Click("Bank Prime")
    }
    else
```

```
    {
        LLARS_Click("Bank Main")
    }

    LLARS_Sleep("Final Sleep", true)
}
```

Use ctx.IsFirst instead of creator-owned first-run globals. The runtime estimator understands this top-level first/following branch.

10. RunCount Runtime Estimation

RunCount automation does not begin with an empty or guessed loop-1 countdown. Before loop 1 starts, LLARS statically inspects the script editing region and builds a timing model from standardized timing calls.

Starting model

- The estimator uses a fixed 1000-run mathematical baseline: one first-loop estimate plus 999 following-loop estimates.
- The user does not execute 1000 warm-up loops. This is only the model used to stabilize the starting average.
- LLARS_Sleep("Literal Section") contributes the configured min/max average for that section.
- LLARS_RandomSleep() contributes the probability-weighted expected value from the shared Random Sleep configuration.
- Top-level ctx.IsFirst / else timing is separated into first-loop and following-loop paths.
- Literal section names are important. Dynamic section-name construction cannot be resolved reliably before execution.

During the run

- Ordinary sleeps leave the current-loop display predictive.
- At LLARS_Sleep("Section", true), the actual randomized final-sleep duration is known, so the current-loop countdown becomes exact.
- Total remaining time becomes exact current-loop remainder plus predicted/measured future loops.
- Following-loop measurements refine the future-loop model as those loops complete.
- The first loop is never reused as the loop-2 prediction when first-run behavior differs.

> **Estimator requirement:** If LLARS cannot build a valid first/following timing model, a RunCount start is blocked with an Estimate Error instead of beginning with misleading timing. Keep estimator-recognizable sleeps on every execution path that needs them.

11. RuneScape Focus and Input Safety

When a run starts, LLARS captures the exact RuneScape window used for that run. Most game-facing Creator API actions reclaim and verify that same HWND before they operate; pixel/color reads are passive and wait for it to be foreground.

What reclaims RuneScape

- Configured clicks and NaturalClick movement/click dispatch.
- Configured hotkeys and fixed-key presses.
- Managed timer callbacks when they fire.
- LLARS_Sleep()/Random Sleep when the configured wait finishes.
- Shared completion actions such as Logout when enabled.

What does not reclaim RuneScape

• Pixel/color reads through the Creator API; they wait for RuneScape to be foreground.
• RunCount loop bookkeeping and status-only work.
• Developer Mode passive inspection.

During waits, the user can work in another application. LLARS_Sleep()/Random Sleep restore RuneScape when they finish; pixel reads wait passively for the user to return. Pause or Exit stops focus reclaim.

Custom raw game-facing code

If a current script genuinely needs a raw AutoHotkey operation that the Creator API does not wrap, call the focus gate immediately before the physical action:

```
if !LLARS_WaitForRuneScape("Custom action")
    return

; raw game-facing action here
```

12. Clicks and NaturalClick

Configured click

```
LLARS_Click("Action Location")
LLARS_Click("Action Location", "right")
```

LLARS_Click() resolves the configured point/area, restores RuneScape, and dispatches through NaturalClick(). AHI is the recommended/default LLARS input backend; Creator API scripts need no AHI-specific code.

NaturalClick behavior

- NaturalClick moves toward one intended target pixel using naturalized movement.
- NaturalClick caps each internally calculated path at 3000 ms; user input can extend the real elapsed time.
- It has no fixed retry limit: cursor displacement starts a new path toward the same intended target.
- The click occurs only after exact-target verification; focus is checked during movement and before the click.

• If Developer Mode is open, LLARS shows the configured coordinate range during the click and for about 500 ms after a successful click. Aborted clicks hide it immediately.

Already-resolved coordinates

```
NaturalClick(x, y)
NaturalClick(x, y, "right")
```

Use direct NaturalClick only when the script already has meaningful dynamic coordinates. Use LLARS_Click() when the destination comes from a Config.ini coordinate section.

13. Keyboard Input

User-configurable RuneScape hotkey

```
LLARS_PressHotkey("Action Hotkey")
```

Purposeful fixed key

```
LLARS_PressKey("Space")
LLARS_PressKey("Enter")
LLARS_PressKey("4")
LLARS_PressKey("WheelDown")
```

Hardcoded keys are not automatically a problem. If Space, Enter, 1, 2, 3, 4, or another key is part of the script design rather than a user choice, keep it fixed and use LLARS_PressKey(). Do not turn every purposeful key into a Config.ini setting.

Both keyboard helpers verify RuneScape focus and automatically use AHI when active; Standard AutoHotkey is the fallback.

14. Status and Activity Text

```
LLARS_SetStatus("Banking")
LLARS_SetStatus("Waiting", "Portable")
```

Use status text to make the main GUI and Developer Mode easier to understand while the script runs. The first argument is the state/action text; the optional second argument replaces the activity/script-name text for that moment.

Status calls do not themselves reclaim RuneScape. They are UI/bookkeeping operations, not game-facing input.

15. Sleeps and Random Sleep

Configured wait

```
LLARS_Sleep("Sleep Short")
```

True final RunCount wait

```
LLARS_Sleep("Sleep Craft", true)
```

Use final=true only on the actual final deterministic configured sleep for that RunCount execution path. This is the handoff from predictive current-loop timing to the exact randomized countdown.

Shared Random Sleep

```
LLARS_RandomSleep()
```

Random Sleep is controlled by the root LLARS Config.ini. LLARS owns its enabled state, chance roll, duration, GUI countdown, counters, logging, and RunCount estimate adjustment.

> Note: During LLARS_Sleep()/Random Sleep, the user may Alt-Tab normally. When the wait finishes, LLARS restores the selected RuneScape client before continuing.

16. Managed Callback Timers

One shot

```
TimerID := LLARS_TimerOnce("Action Timer", Func("Action"))
```

Repeating random timer

```
TimerID := LLARS_TimerRepeat("Action Timer", Func("Action"))
```

LLARS_TimerRepeat() is implemented as chained one-shot callbacks. After every callback, a fresh interval is randomized from the configured min/max range.

Stop timers

```
LLARS_TimerStop(TimerID)
LLARS_TimerStopAll()
```

- Managed timers stop automatically during framework cleanup.
- When a managed callback becomes due, LLARS reclaims/verifies RuneScape before entering the callback.
- If a callback ends the run or stops itself, it is not rescheduled.
- Use Func("Name").Bind(...) when a callback needs arguments.

> **Note:** LLARS_TimerSchedule() still exists as a label-based compatibility bridge. New code should use LLARS_TimerOnce()/LLARS_TimerRepeat().

17. Timer, MultiTimer, and UntilDone Scripts

Timer templates still use LLARS_StartTimerRun() for the user-selected overall duration. The template owns the shared countdown and completion path; creator code normally schedules operational work with managed callbacks.

```
TimerSetup()
{
    global ScriptTimerID
    ScriptTimerID := LLARS_TimerRepeat("Script Timer", Func("ScriptTimer"))
}

ScriptTimer()
{
    LLARS_SetStatus("Working")
    LLARS_Click("Action Location")
}
```

MultiTimer

```
MultiTimerSetup()
{
    global TimerOneID, TimerTwoID
    TimerOneID := LLARS_TimerRepeat("Timer One", Func("TimerOne"))
    TimerTwoID := LLARS_TimerRepeat("Timer Two", Func("TimerTwo"))
}

TimerOne()
{
```

```
    LLARS_Click("First Location")
}

TimerTwo()
{
    LLARS_PressHotkey("Action Hotkey")
}
```

Keep the template-owned countdown outside creator logic. UntilDone scripts set LLARS_SCRIPT_TYPE := "UntilDone" and use LLARS_StartUntilDoneRun()/LLARS_EndUntilDoneRun(); they do not prompt for a duration.

18. Pixel and Color APIs

Read one configured pixel

```
currentColor := LLARS_PixelColor("Pixel Coordinate")
```

Match one target

```
if LLARS_PixelMatches("Pixel Coordinate", "Target Color")
{
    LLARS_Click("Action Location")
}
```

Match any of several targets

```
colors := ["Target Color One", "Target Color Two", "Target Color Three"]
matched := LLARS_PixelMatchesAny("Pixel Coordinate", colors)

if (matched != "")
{
    LLARS_SetStatus("Detected", matched)
    LLARS_Click("Action Location")
}
```

LLARS_PixelMatchesAny() samples the watched pixel once and compares it against the enabled color sections. It returns the matching section name or an empty string.

Pixel helpers use client coordinates and sample only when the selected RuneScape window is already foreground. If the user Alt-Tabs away, they wait rather than activating or restoring RuneScape.

19. Optional Features and Dependencies

Simple option

```
[Optional Action]
option=false

if LLARS_ConfigReadBool("Optional Action")
{
    ; optional behavior
}
```

Typed optional action

```
[Optional Hotkey]
option=false
type=hotkey
hotkey=
```

Typed helpers and action helpers automatically respect option=false where appropriate. LLARS_ConfigEnabled() also honors dependency chains.

Dependency

```
[Advanced Feature]
option=false

[Advanced Hotkey]
depends=Advanced Feature
type=hotkey
hotkey=
```

When the dependency is disabled, Configuration Status and Developer Mode treat the dependent typed setting as Not Required.

20. Developer Mode

Developer Mode is a live framework diagnostic view. It is optional and must never be required for the script to work.

Full mode layout

- Two centered run-summary rows: State / Run Type / Progress, then Elapsed / Final Sleep.
- Live Inspector on the left and Active Hotkeys on the right.
- Configuration Diagnostics full width beneath the top panes.
- Recent Framework Actions full width and given the largest action-history area.
- For a Timer run, Progress shows the same live remaining-time countdown visible in the main LLARS GUI.

Live Inspector

- Game Status, Mouse POS X, Mouse POS Y, and Mouse RGB (the color under the mouse).
- For every enabled typed color target: section name, Pixel Coordinate, Current Pixel, and Pixel Target.
- The pane is scrollable and preserves its scroll position while live values refresh.

Click overlay

When Full or Lightweight Developer Mode is open, the active coordinate range is shown during NaturalClick and for about 500 ms after a successful click. Aborted clicks hide it immediately.

Lightweight mode and hotkey toggle

Lightweight Mode uses the same window width and shows Recent Framework Actions only. The Developer Mode hotkey toggles whichever view is currently selected. If Lightweight is selected, repeated hotkey presses open/close Lightweight until the Full Mode button switches the preference back to Full.

21. Logging and Diagnostics

```
Log("EVENT", "Message")
```

Use the shared logger for task-specific events that are useful when diagnosing a script. Creator API operations also populate Recent Framework Actions automatically.

Developer Mode is not a full AutoHotkey execution trace. Raw commands that bypass LLARS may not appear with the same labels or detail. That is one reason the current docs prefer Creator API actions for normal behavior.

Configuration errors

Typed config helpers route invalid runtime reads through the existing CONFIG ERROR logging path and Developer Mode. Startup validation remains the primary check before a run begins.

22. Testing a Script

- Run Configuration Status and confirm required typed values are Ready.
- Test Start, Pause, Resume, and Exit.
- For RunCount, test run counts 1 and 2 so first/following behavior is exercised.
- Verify the estimate exists before loop 1 and that final=true produces the exact current-loop countdown.
- Alt-Tab during LLARS_Sleep() and verify RuneScape is restored when the sleep finishes.

- Open Developer Mode and verify Active Hotkeys, Live Inspector, Configuration Diagnostics, action history, and the temporary coordinate overlay.
- For managed timers, confirm they stop on completion/Exit and do not leak into the next run.
- For multi-color scripts, verify the reset condition prevents repeated triggering on an unchanged pixel.
- For UntilDone, verify its script-defined condition ends the run and optional Logout still works.

> **Note:** A finished script must work correctly with Developer Mode closed. Developer Mode should reveal behavior, not supply behavior.

23. Converting an Older Script

Preserve the task logic first. Convert the shared mechanics around it.

1. Keep the original action order, purposeful fixed keys, task-specific state, and Config.ini meaning.
2. Replace manual RunCount lifecycle with LLARS_RunCount(Func("Run")).
3. Replace firstrun logic with ctx.IsFirst.
4. Replace configured IniRead + Random + NaturalClick blocks with LLARS_Click().
5. Replace configured IniRead + Random + Sleep blocks with LLARS_Sleep(); mark the actual final RunCount sleep with final=true.
6. Replace user-configured Send calls with LLARS_PressHotkey(); preserve intentional fixed keys through LLARS_PressKey().
7. Replace ordinary pixel boilerplate with LLARS_PixelColor()/LLARS_PixelMatches()/LLARS_PixelMatchesAny().
8. Replace creator-owned action timers with LLARS_TimerOnce()/LLARS_TimerRepeat() where their callback model fits.
9. Keep unusual custom calculations/custom movement only when the behavior truly needs them, and place LLARS_WaitForRuneScape() immediately before raw game-facing operations.
10. Test one representative script of each pattern before mass-converting the rest.

The Legacy Guide documents the old manual patterns when a script intentionally needs to remain low-level.

24. Creator API Reference

API	Purpose
LLARS_RunCount(callback)	Current RunCount lifecycle around one callback.
LLARS_RunActive(requireRuneScape := false)	Check active, unpaused run state; optionally require active

	RuneScape.
LLARS_WaitForRuneScape(action)	Restore/verify the exact selected RuneScape window before custom game-facing work.
LLARS_Click(section, button := "left")	Resolve configured point/area and NaturalClick it.
LLARS_PressKey(key)	Send a purposeful fixed key safely.
LLARS_PressHotkey(section)	Read/send a configured hotkey safely.
LLARS_Sleep(section, final := false)	Randomized configured wait; final=true performs exact RunCount handoff.
LLARS_RandomSleep()	Run the shared optional Random Sleep behavior.
LLARS_TimerOnce(section, callback)	Managed one-shot randomized callback timer.
LLARS_TimerRepeat(section, callback)	Managed repeating timer with a fresh random interval every cycle.
LLARS_TimerStop(ref)	Stop one managed timer.
LLARS_TimerStopAll()	Stop all Creator API timers.
LLARS_SetStatus(status, activity := "")	Update shared LLARS status/activity text.
LLARS_PixelColor(section)	Read RGB at configured point.
LLARS_PixelMatches(point, color)	Compare one point to one configured color.
LLARS_PixelMatchesAny(point, colors)	Compare one sampled point to several color sections.
LLARS_ConfigRead*	Typed/config-specific readers for task logic.
LLARS_ConfigEnabled(section)	Check option/dependency-enabled state.
NaturalClick(x, y, button := "left")	Low-level natural click for already-resolved dynamic coordinates.

25. Framework Presence and Includes

Maintained scripts keep a local LLARS_FrameworkAvailable() check and a series of optional bootstrap includes at the end of the file. This lets a script exist at different folder depths under Scripts while still locating Core\LLARS_ScriptBootstrap.ahk.

Keep this template scaffolding intact. A single .ahk opened directly from inside a ZIP/7Z/RAR without the full project should show the framework error instead of failing unpredictably.

> **Note:** Do not copy Core function bodies into individual scripts. Call the shared functions. This avoids duplicate-function collisions and keeps future framework fixes centralized.

26. Common Creator Mistakes

Rebuilding a RunCount loop

New code should not own runcount/firstrun bookkeeping or LLARS_BeginLoop()/LLARS_EndLoop(). Use LLARS_RunCount().

Using raw Send/Click for normal actions

That bypasses centralized focus safety and diagnostics. Use LLARS_PressKey(), LLARS_PressHotkey(), and LLARS_Click().

Turning purposeful fixed keys into configuration

If Space or another key is intentionally part of the script behavior, keep it fixed and call LLARS_PressKey(). Config.ini is for user choices, not every constant.

Dynamic sleep section names in RunCount

The pre-run estimator requires literal section names for standardized LLARS_Sleep() parsing. Keep timing visible and deterministic inside the editing region.

Calling raw actions without a focus gate

For custom raw game-facing work, call LLARS_WaitForRuneScape() immediately before the physical operation. This keeps the action safe even if other logic ran after a configured wait.

Duplicating or breaking editing markers

There must be exactly one begin marker and one end marker in the canonical three-line separator format.

27. About LLARS

LLARS stands for Local Library of Automated Routines & Scripts.

The LLARS project, framework, scripts, templates, and documentation are created and maintained by Gubna. LLARS is free and open source.

GitHub: https://github.com/Gubna-Tech/RuneScape

28. The Simple Version

- Start from a maintained template.
- Keep task code inside the exact editing markers.
- RunCount: LLARS_RunCount(Func("Run")); use ctx.IsFirst for first-run differences.
- Clicks: LLARS_Click(). Fixed keys: LLARS_PressKey(). Configured hotkeys: LLARS_PressHotkey().
- Waits: LLARS_Sleep(); mark the true final RunCount wait with final=true.
- Timers: LLARS_TimerOnce()/LLARS_TimerRepeat().
- Pixels: LLARS_PixelMatches() or LLARS_PixelMatchesAny().
- Use LLARS_WaitForRuneScape() immediately before any unavoidable raw game-facing action.
- Configured sleeps allow Alt-Tabbing, then restore RuneScape when they finish.
- UntilDone: set LLARS_SCRIPT_TYPE := "UntilDone" and use the UntilDone start/end helpers.
- Use the Legacy Guide only when you intentionally want traditional/manual AHK patterns.

> **Note:** Goal: creator code says what the task does. LLARS handles the shared framework around it.